Project: Productionizing your LLM Server

Due: 02/14/2026 11:59 PM
Extended Deadline: 02/15/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Refactor to FastAPI
- Task Overview
- Task

Testing and pre-commit hooks
- Task Overview
- Task

CI pipeline for FastAPI
- Task Overview
- Task

Wrap Up
- Conclusion

Testing and pre-commit hooks (Task)

Confidence Amplification

In the previous task, we built our first FastAPI application. The purpose was to leverage features provided by FastAPI to increase the usability (via comprehensive API documentation) and speed (via asynchronous programming) of our application. In this task, we will examine concepts that generally apply to Python programs of any size and help us verify our code quality further.

This is also our preparation step for the CI pipeline, where we will enforce these checks. Specifically, we are focusing on the following attributes of our code:

1. **Documentation**: There is consistent and adequate documentation provided within the code.
2. **Styling**: The styling across the code is consistent.
3. **Testing**: The code has automated tests.

Currently, we don't have a CI system—that's fine. We can still ensure that what we commit meets our quality standards. To accomplish that, we will use pre-commit hooks.

Setup: Git-ing Hooked on Hooks

Before CI/CD systems like Travis CI, GitHub Actions, and GitLab Runners, Git needed a way to assist developers in providing these checks. Since Git was designed to be a distributed versioning system without a central source of truth, it needed something that could be easily run locally without consulting an existing server.

The solution that the original users of Git developed was the hook—a simple script that resides under `.git/hooks/` and activates when a specific condition is triggered.

There are a wide variety of conditions upon which a Git hook can run, but the most popular are:

pre-commit, which runs before files are committed to the Git history.
pre-push, which runs before files can be pushed to a Git repository.

In general, they are quite powerful, supporting workflows beyond the more traditional ones we discussed in Cloud DevOps, such as patch-based workflows or email-based workflows like those used by the Linux Kernel development team.

The main drawbacks of using Git Hooks when you have access to systems like GitHub Actions or GitLab Runners are:

Added Complexity: Git Hooks are written as bash scripts, which can make keeping everything organized quite challenging. It's usually difficult to ensure that all scripts are running and failing when you want them to, especially if you lack strong command-line experience with your terminal of choice.
Easy to Disable: When working with a larger team on a centralized repository, it makes sense to use CI, at least to ensure that your build is centralized and that standards are completely enforced. As you will see, Git Hooks can be easily disabled. This means you cannot trust that your style checks are actually being enforced beyond running them yourself and then using `git blame` to identify who broke them.
Can Lead to Bad Practices: If you use too many pre-commit or pre-push hooks, it becomes easy to forget that you need to commit your code. Since we want to commit early and often, too many checks can run counter to good workflow habits.

For this task, since the goal is to introduce tools and have you iterate on the provided code, we will use pre-commit, a Python package specifically built to manage and install hooks. It will handle the complexities of using them for us, so we can focus on using them in this context to improve our code quality.

To install the `pre-commit` package, you must use Poetry:

```
poetry add --group dev pre-commit
```

Now we need to create a `.pre-commit-config.yaml` file, which will contain the commands that need to run when the

pre-commit hook is triggered. Here, the code that runs when the hook is triggered can be in our repository, or we can map it to an external GitHub repository.

This is important because we can now use already existing hooks for common use cases like linting and type checking.

Create the `.pre-commit-config.yaml` file in your project's **root folder** and copy the following content into it:

```
repos:
  - repo: https://github.com/astral-sh/ruff-p
    rev: v0.14.5
    hooks:
      - id: ruff-check
        args: [--fix, --output-format=concise
      - id: ruff-format
  - repo: https://github.com/jsh9/pydoclint
    rev: 0.8.1
    hooks:
      - id: pydoclint
        args: [--style=google]
```

Observe that we use external repositories that provide these hooks. What these hooks are doing will be explained later in this task.

Now, run the install command to install these hooks into the `.git/hooks` folder. This tells Git to automatically run these before every commit:

```
poetry run pre-commit install
```

To actually download and run the hooks, we can use the following command, or you can perform a commit which will trigger them:

```
poetry run pre-commit run --all-files
```

Information

By default, pre-commit hooks run only on files that have changed since the last commit.

Subtask 1: Linting is Ruff

Linting is a process that checks your code and identifies sections that violate a certain set of rules. These violations

are related to syntax checks, poor coding practices, and sometimes common security vulnerabilities. In our `.pre-commit-config.yaml`, we installed **Ruff**, which provides very fast linting and code formatting for Python and is highly customizable. You can glance over the complete list of rules that Ruff can enforce here.

Before Ruff can report any violations, we need to instruct it and explicitly tell it what kind of violations we are interested in. To do that, you need to create a `ruff.toml` file in the project root directory.

Add the following to your `ruff.toml` file:

```
[lint]
select = ["F", "D", "S", "RET", "EM", "T20",
ignore = ["S101"]
[lint.pydocstyle]
convention = "google"
```

Here, each element in the list denotes a type of violation we want Ruff to report. For example:

> **F**: Pyflakes, which focuses on providing more technical best-practice changes.
> **D**: Pydocstyle, which simply checks if documentation exists or not.
> **S, RET, EM, T20, FA**: Flake8, which is a style and feature checker. It's also one of the most opinionated tools out there for Python, so we will not worry about all of the rules it puts in place.

Now, let's see for the first time how many violations Ruff reports. Run the following command to do that:

```
poetry run pre-commit run --all-files ruff-ch
```

You would see an output something like the below, but not exactly the same:

```
ruff check.................................
- hook id: ruff-check
- exit code: 1

llm_server/llm_server/__init__.py:1:1: D104 M
llm_server/llm_server/main.py:1:1: D100 Missi
llm_server/llm_server/main.py:12:7: D101 Miss
llm_server/llm_server/main.py:16:9: D102 Miss
llm_server/llm_server/main.py:20:17: EM101 Ex
```

```
llm_server/llm_server/main.py:25:7: D101 Miss
llm_server/llm_server/main.py:32:7: D101 Miss
llm_server/llm_server/main.py:38:7: D101 Miss
llm_server/llm_server/main.py:48:11: D103 Mis
llm_server/llm_server/main.py:53:11: D103 Mis
llm_server/llm_server/main.py:63:12: RET504 U
Found 11 errors.
No fixes available (2 hidden fixes can be ena
```

The above output should be enough to tell you what Ruff wants you to do and in which line. Feel free to search for the specific rule code (e.g., D101, RET504, etc.) on Ruff's website to learn more about it.

Your task is to modify the repository's source code until all of these errors disappear.

You are expected to repeatedly run the above command and keep eliminating more violations until there are none left and you see something like:

```
ruff check.................................
```

You can now run the Ruff formatter as well, which automatically formats the code for you—removing unnecessary spacing, reducing line length, and many such small violations. It is guaranteed that the changes done by the formatter won't change your application's functionality or break anything.

To run the formatter, run the command:

```
poetry run pre-commit run --all-files ruff-fo
```

You might see the formatting fails. This is expected because the Ruff formatter would first report the violations and then fix them. Once you run the same command again, it should pass, and you would see:

```
ruff format................................
```

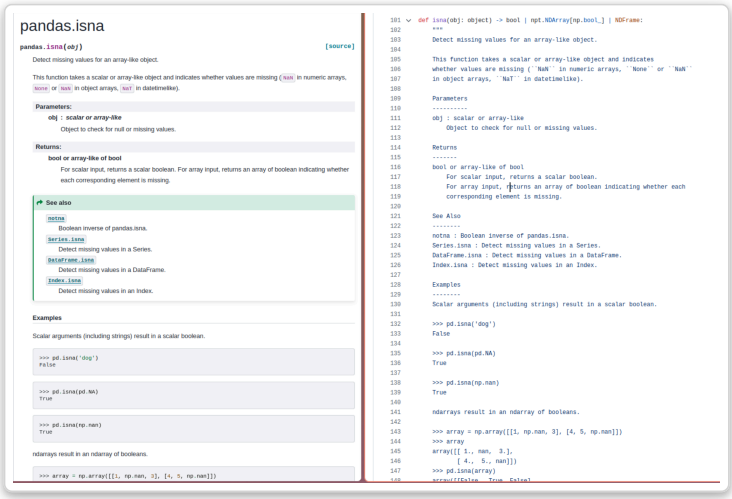
If you don't see this, we expect you to read the violations and fix any pending issues until your ruff-format passes without errors.

Subtask 2: Documenting Your Code

In Python, a docstring is a string literal that occurs as the first statement in a module, function, class, or method

definition. Python automatically populates the `__doc__` attribute of the object based on the docstring. They are used to document your module, class, functions, or methods so other programmers can understand your intent. Based on this docstring, tools like Sphinx can automatically generate an HTML version of documentation.

Observe in the following image that `pandas` used docstrings to tell us what the function `isna` does. As you can see, the docstring is specified in the code where `isna` was implemented, and you can see the same docstring on the HTML version. In addition to this HTML website, docstrings also enable the hover feature in VSCode, so when you are working with pandas, you can hover over the `isna` function and see the same docstring.



In the previous task, Ruff made us add docstrings to our files, classes, and functions. But it doesn't verify whether those docstrings are correct. Here is an example to explain this issue:

```
def testme(blah: str) -> None:
    """Check if argument is equal to 'hi'.

    Args:
        blah (list): Input Argument.

    Returns:
       int: Whether testme is 'hi' or not.
     """
     pass
```

Here, we have a fairly well-written docstring. It follows all of pydocstyle's conventions, making sure that the comments follow Python's established best practices, but the comment does not describe the code in question:

It states the function performs a test that it doesn't.
It states that there is a return value when the function returns nothing.

This is especially dangerous if someone automatically generates documentation without looking at the code. However, if the docstrings are wrong, then there is a high possibility that someone will try to use the code and fail dramatically. We need tools like pydoclint to ensure that when we refactor code, we can trust that any mistakes like this can be caught early.

In order to understand what a comment is dictating, pydoclint requires you to pick one of a few different kinds of docstring formats. These are conventions people use to specifically make docstrings consistent and standardized, as well as to make automatic documentation generation far easier.

In the past, when Sphinx, the standard tool for documentation generation, did not support other docstring formats, Python code used the "reST" style:

```
def hi(param1, param2):
    """
    This is a reST style.

    :param param1: this is a first param
    :param param2: this is a second param
    :returns: this is a description of what i
    :raises keyError: raises an exception
    """
```

It's functional, but it makes it difficult to read at a glance and does not separate arguments from function outputs. This makes it challenging to read at a glance, even if it allows for automatic documentation generation.

Nowadays, people either choose Google or NumPy style docstrings.

NumPy style docstrings look like the following snippet:

```
def hi(first, second, third):
    """
    My numpydoc description of a kind
    of very exhaustive numpydoc format docstr

    Parameters
    ----------
```

```
    first : array_like
        the 1st param name `first`
    second :
        the 2nd param
    third : {'value', 'other'}, optional
        the 3rd param, by default 'value'

    Returns
    -------
    string
        a value in a string

    Raises
    ------
    KeyError
        when a key error
    OtherError
        when an other error
    """
```

At a glance, this is significantly more readable. You can see spaces between everything, and while the format is a bit bulky, it's just easier to look as compared to reST.

While NumPy docstrings are great, we will use Google style docstrings, as they are also well-liked and are generated by default by a really nice VSCode extension that helps you generate docstrings with a type-annotated function.

They look like the following:

```
def hi(param1, param2):
    """
    This is an example of Google style.

    Args:
        param1: This is the first param.
        param2: This is a second param.

    Returns:
        This is a description of what is retu

    Raises:
        KeyError: Raises an exception.
    """
```

They are a bit more compact vertically compared to NumPy docstrings, at the cost of potentially writing less. In general,

the style of your docstrings does not matter so long as you are consistent about it amongst developers working on the same repository.

For this class, it's just important for us that we tell you there is a difference and that you need to pick one to use pydoclint in other projects, as otherwise it will be unable to interpret the docstrings effectively.

For this subtask, all you need to do is fix all of the warnings generated by pydoclint.
Like Ruff, we can run pydoclint separately using:

```
poetry run pre-commit run --all-files pydocli
```

Subtask 3: Testing and Mocking FastAPI

Now that you have code that is well-documented and well-styled, we can turn to our third goal: making our code well-tested. Practically speaking, we will aim for 100% code coverage for this task.

As we have repeated in Cloud Devops, code coverage, by itself, does not correlate with test quality. You can get 100% code coverage through a wide variety of means, as the only indicator of coverage is whether the line in question has been run or not. Practically, when you work on higher-impact code, you should aim for far higher bars than code coverage, such as making user stories and performing contract testing.

First, we use Poetry to install some packages required for testing:

```
poetry add --group dev coverage pytest httpx
```

In our case, these packages include:

1. **coverage**: Used to calculate code coverage.
2. **pytest**: Our testing framework.
3. **httpx**: Used implicitly in FastAPI's testing system.

First, we need to create a tests module which would house all our test cases. To do that, create a folder called `tests` and files `__init__.py` and `test_requests.py` inside it. Remember, adding an `__init__.py` makes this folder a Python module. Your directory structure should now look like:

```
cloud-native-handout-<github-username>
├── README.md
├── answers
│   └── ....
├── llm_server
│   ├── Makefile
│   ├── README.md
│   ├── docker
│   │   └── Dockerfile
│   ├── k8s
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── llm_server
│   │   ├── __init__.py
│   │   └── main.py
│   ├── poetry.lock
│   ├── poetry.toml
│   ├── pyproject.toml
│   └── tests
│       ├── __init__.py
│       └── test_requests.py
├── ruff.toml
├── state.json
├── submitter
├── terraform
│   ├── main.tf
│   └── variables.tf
├── terraform_init.log
└── tree.txt
```

Add the following code to the `test_requests.py` file:

```
"""API Server Tests."""

from fastapi.testclient import TestClient
from llm_server.main import app, LLMPrompt
from fastapi.exceptions import RequestValidat
from unittest.mock import patch

client = TestClient(app=app)

def test_health_route():
    """Test for health endpoint."""
    response = client.get("/health")
    assert response.status_code == 200

def test_api():
```

```
    # TODO: Mock the response from LLM model
    pass

def test_token_count_happy_path():
    # TODO: Verify that the LLMPrompt doesn't
    # [HINT]
    # - Since llm is just a mock response, ma
    # - Think about what kind of data you wan
    pass
```

Essentially, your task is to complete the TODOs here and achieve 100% code coverage.

In the above test file, there are a few things to observe:

1. `Pytest` automatically finds all files in your project that start with `test_` and looks for **functions** inside those files which start with `test_`. Each function is an individual test case; you can either run a single one, multiple, or all the test cases at once.
2. FastAPI integrates well with Pytest by giving us a utility called `TestClient`. This allows you to simulate HTTP requests to your API endpoints directly within your test suite without running the server or main.py file.
3. The `test_health_route` is completed for your reference. We first create the `client` object where we initialize the TestClient and pass in `app` (our FastAPI application). Make sure that you named your FastAPI application `app` in the `main.py`. Find the line `app = FastAPI()` —that's your FastAPI application. We can simulate a GET request on the "/health" endpoint by calling `client.get("/health")` and verify that the `response.status_code` is `200`. While here we are simply checking the status code, you can check anything that you like, for example, the response body, the response headers, etc.
4. The `assert` statement is the primary mechanism for verifying conditions and validating expected outcomes within your tests. If an assert fails, that means the test case failed.
5. Other imports would become relevant as we progress forward.

Rectifying a mistake

Before we start implementing the test cases, you need to understand and rectify a mistake we made earlier due to convenience.

In your `main.py`, observe how you are initializing your `llm`. You would most probably see a global `llm` variable that creates a new `llm` object like this:

```
llm = Llama(model_path="tinyllama-1.1b-chat-v
```

When we run the server using Uvicorn, this global variable gets initialized and our app works. But this is not the correct way, because of two reasons:

1. What if later you extend your application to multiple files? This variable may never get initialized if its file is not called directly.
2. When testing, we just use the `app` object and not run the entire file, thus the TestClient doesn't know about the `llm` variable.

We can leverage FastAPI's lifespan mechanism to initialize this object at startup. To do that, we use the `asynccontextmanager`. Make the following changes in the main.py:

Add the import:

```
from contextlib import asynccontextmanager
from typing import AsyncGenerator
```

Add the `lifespan` function and a global `LLM_MODEL` variable before you initialize `app`:

```
LLM_MODEL: Llama | None = None


@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncGene
    """Add LLM_MODEL to app context at startu
    global LLM_MODEL
    LLM_MODEL = Llama(
        model_path="tinyllama-1.1b-chat-v1.0.
        n_ctx=512,
        n_batch=1,
    )
    yield
    del LLM_MODEL

def get_llm() -> Llama:
    """Get LLM_MODEL or raise an assertion er
```

```
    assert LLM_MODEL is not None
    return LLM_MODEL
```

When you create your FastAPI app, change it to:

```
app = FastAPI(lifespan=lifespan)
```

Then replace wherever you used the `llm` variable with `get_llm()`. Here's an example, but there are surely more places where `llm` was used:

```
token_cnt = len(get_llm().tokenize(message.en
```

Please checkout the [FastAPI documentation] to understand more about what we did here.

Back to Testing

First, we add a `.coveragerc` file in the parent llm_server directory. This file should be at the same level as the pyproject.toml file. Copy and paste the following into the `.coveragerc` file.

```
[run]
source = .
```

The above script tells Coverage to only measures files from the llm_server directory, instead of anything imported from elsewhere.

Let's run the test file as is and see how much coverage we are starting with. Use the following command to run the test cases and also print a report for coverage:

```
poetry run coverage run -m pytest; poetry run
```

You should see output similar to the following:

```
=============================================
platform linux -- Python 3.12.3, pytest-9.0.1
rootdir: <your root dir>
configfile: pyproject.toml
plugins: anyio-4.11.0
collected 3 items

tests/test_requests.py ...

=============================================
```

```
Name                      Stmts   Miss  Cover
-------------------------------------------
llm_server/__init__.py        0      0   100%
llm_server/main.py           46     10    78%
tests/__init__.py             0      0   100%
tests/test_requests.py       12      0   100%
-------------------------------------------
TOTAL                        58     10    83%
```

Interim Validation

The above command must run without any errors. The coverage number may be different, but if you encounter any errors in running the test cases, you must fix them before moving forward. Please reach out if you are stuck for an extended period of time.

Now, let's look at the above output. The coverage report tells us about the lines that don't have coverage. These line numbers can be different for you. For us, these lines represent the following:

1. **31-32**: This is the `get_llm` function that is never being run in any test case, thus missing from coverage.
2. **46-50**: This is the `validate_length` function in `LLMPrompt`.
3. **92-98**: This is the `get_api` function.

We need to write test cases that cover these functions. We will take this opportunity to introduce you to the concept of `mocks`.

Mocks allow you to replace parts of your system under test with simulated objects, isolating the code you are testing from its dependencies.

Here, our dependency is the LLM Model. You want to test your application irrespective of whether the model file is present or not. In our case, the dependency is a file; it could have been a database connection or a third-party API, in which case it is almost impossible to carry that dependency around in all environments. We can mock this dependency—that means we can provide the same output to our application as it would have gotten had the dependency been there.

To mock the `LLM_MODEL`, you can write something like this in the `test_api` function:

```
def test_api():
    with patch('package.module.varname') as l
        llm.return_value = {"id":"cmpl-f1737a
```

Where `package.module.varname` is something you will need to replace. What this does is the following:

> When we import or use `varname` from `package.module` anywhere that is called from within the `with` statement, `varname` is replaced by `llm` instead.
> Setting `llm.return_value` to be the string here makes `llm` appear to be a function, which will always return this object, no matter the arguments.

Thus, by using mock, we can pretend that we have a model to test with, even if we run this test on a CI pipeline that can't run expensive model inference.

With this knowledge, you should now be able to finish implementing both the test cases and achieve 100% test coverage.

Checking Quality Once Again

After you have achieved 100% test coverage, re-run ruff-check, ruff-format, pydoclint, and mypy and fix all the violations you see.

Here are the commands to run these:

```
poetry run mypy -p llm_server --strict
poetry run pre-commit run --all-files ruff-ch
poetry run pre-commit run --all-files ruff-fo
poetry run pre-commit run --all-files pydocli
```

You would surely see a few violations at least in the newly added test files. Make sure all four of the tools are happy and have no violations.

Submission

```
./submitter -t task2
```

Make sure to also commit the code to your GitHub.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform